
Topic 1 – Lost & Found Matching Service

SWE Final Project Team

Mahammad Javadzade ■ Teymur Alasgarov ■ Parvin Salahov

Final Project Defense – 2026-09-20

<https://github.com/ParvinSalahov/SWE-final-proj>

The problem

What the system does

Users report lost or found items with image and text, and the system ranks likely matches using VLM output plus embedding similarity.

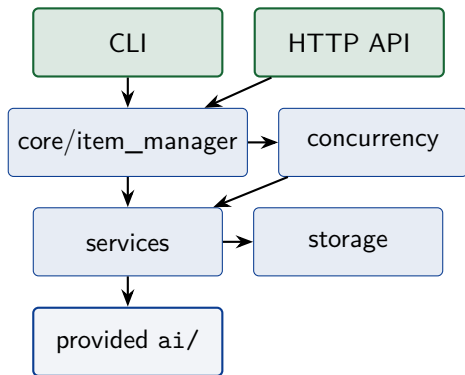
Inputs

- ▶ item metadata: status, description, colors, attributes
- ▶ optional image file

Outputs

- ▶ structured item record
- ▶ embedding and match score
- ▶ ranked list of likely matches

Architecture



Design decisions

1. **Bounded async concurrency** – we used `asyncio.Semaphore` to keep provider and DB calls under control.
2. **Service boundary around AI** – retries, validation, and caching are kept out of the UI layer.
3. **Filesystem + database split** – binary image data stays as files, metadata stays in the repository.

Concurrency benchmark

Workload: batch of 10 mock item registrations.

	Sequential	Concurrent
Wall-clock (s)	0.5028	0.1009
Speedup	1.0×	4.98×

Why it matters: the parallel design is a net win for bursty processing, while the semaphore keeps the work polite to external providers.

Failure-mode story

Scenario: malformed or oversized image uploaded before AI description step.

- ▶ image is checked for length, magic headers, and Pillow decode validity
- ▶ invalid/unsafe input is rejected before the provider call
- ▶ no silent backend crash, no partial record persisted

Result: clearer logs, better user feedback, and a service that fails early and explicitly.

Docker demo

Build the app:

```
$ docker build -t lost-found-final .  
$ docker run --rm --env-file .env lost-found-final python scripts/demo.py
```

Start the database stack:

```
$ docker compose up -d  
$ docker compose exec app python scripts/demo.py
```

Testing and type checks

- ▶ `pytest -q` 82 passed in 1.44s
- ▶ `tests/test_ai_smoke.py` 27 passed
- ▶ `mypy src tests scripts -python-version 3.12` no errors
- ▶ `pyright src scripts tests` 0 errors, 0 warnings

Limitations and next steps

- ▶ benchmark isolates concurrency logic rather than full external-provider latency
- ▶ single-instance service, not a full distributed deployment
- ▶ rate-limit handling is local and retry-based, not queue-based

If we had more time: add richer observability, larger batch load tests, and a multi-provider failover strategy.

Team contributions

Member	Primary work	Share
Mahammad Javadzade	storage, concurrency, benchmarking	30%
Teymur Alasgarov	API + CLI + service flow	38%
Parvin Salahov	tests, provider integration, validation	32%

Questions?

SWE Final Project Team

<https://github.com/ParvinSalahov/SWE-final-proj>

"We can walk through any file in the repo."